



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 从用户态到内核态

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 B202

实验时间: 2025/06/10

Section 1 实验概述

在本章中，我们首先会简单讨论保护模式下的特权级的相关内容。特权级保护是保护模式的特点之一，通过特权级保护，我们区分了内核态和用户态，从而限制用户态的代码对特权指令的使用或对资源的访问等。但是，用户态的代码有时不得不使用一些特权指令，如输入输出等。因此，我们介绍了系统调用的概念和如何通过中断来实现系统调用。通过系统调用，我们可以实现从用户态到内核态转移，然后在内核态下执行特权指令等，执行完成后返回到用户态。在实现了系统调用后，我们通过三步来创建了进程。这里，我们需要重点理解我们是如何通过分页机制来实现进程之间的虚拟地址空间的隔离。最后，我们介绍了 fork/wait/exit 的一种简洁的实现思路。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
 - 虚拟机版本/处理器型号：Virtualbox
 - 代码编辑环境：Vscode+nasm+C/C++插件
 - 代码编译工具：gcc/g++ （64 位）
 - 重要三方库信息：

Section 3 实验任务

- 实验任务 1：实现系统调用的方法
- 实验任务 2：进程的创建和调度
- 实验任务 3：fork 的实现
- 实验任务 4：wait & exit 的实现

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

● 任务要求：实现系统调用的方法

复现指导书中“第一个进程”一节，并回答以下问题：

1. 请解释为什么需要使用寄存器来传递系统调用的参数，以及我们是如何在执行 `int 0x80` 前在栈中找到参数并放入寄存器的
2. 请使用 `gdb` 来分析在我们调用了 `int 0x80` 后，系统的栈发生了怎样的变化？`esp` 的值和在 `setup.cpp` 中定义的变量 `tss` 有什么关系？此外还有哪些段寄存器发生了变化？变化后的内容是什么？
3. 请使用 `gdb` 来分析在进入 `asm_system_call_handler` 的那一刻，栈顶的地址是什么？栈中存放的内容是什么？为什么存放的是这些内容？
4. 请结合代码分析 `asm_system_call_handler` 是如何找到中断向量号 `index` 对应的函数的
5. 请使用 `gdb` 来分析在 `asm_system_call handler` 中执行 `iret` 后，哪些段寄存器发生了变化？变化后的内容是什么？这些内容来自于什么地方？

● 实验步骤：

1. 请解释为什么需要使用寄存器来传递系统调用的参数，以及我们是如何在执行 `int 0x80` 前在栈中找到参数并放入寄存器的？

1.1 使用寄存器来传递系统调用的参数

用户程序使用系统调用时会进行特权级转移，如果我们使用栈来传递参数，在我们调用系统调用的时候，系统调用的参数（即 `asm_system_call` 的参数）就会被保存在用户程序的栈中，也就是低特权级的栈中。

系统调用发生后，我们从低特权级转移到高特权级，此时 CPU 会从 TSS 中加载高特权级的栈地址到 `esp` 寄存中。而 C 语言的代码在编译后会使用 `esp` 和 `ebp` 来访问栈的参数，但是前面保存参数的栈和现在期望取出函数参数而访问的栈并不是同一个栈，因此 CPU 无法在栈中找到函数的参数。

1.2 执行 `int 0x80` 前在栈中找到参数并放入寄存器

在用户态，系统调用通过封装函数 `asm_system_call` 发起，该函数会按照预定的调用约定将系统调用号放入 `eax`，并将最多五个参数分别放入 `ebx`、`ecx`、`edx`、

esi、edi 等通用寄存器中。然后，执行 `int 0x80` 指令触发软中断，CPU 会自动将当前执行现场压入内核栈，并跳转到内核中断描述符表中注册的 0x80 号处理函数。

进入内核态后，`asm_system_call_handler` 被调用，它首先保存现场，包括段寄存器和通用寄存器。由于进入中断后只有代码段（cs）被 CPU 自动切换，而数据段（如 ds, es, gs 等）依然保留用户态的值，因此需要手动设置这些段寄存器为内核的数据段选择子，确保访问内核数据是安全合法的。

随后，处理函数将 ebx 到 edi 寄存器中的参数依次 push 到当前内核栈上，为后续调用系统调用处理函数做准备。系统调用号保存在 eax 中，作为索引用于查找系统调用函数表 `system_call_table`，并使用 `call` 指令调用对应的 C 函数，这些参数会作为 C 函数的入参从栈中读取。

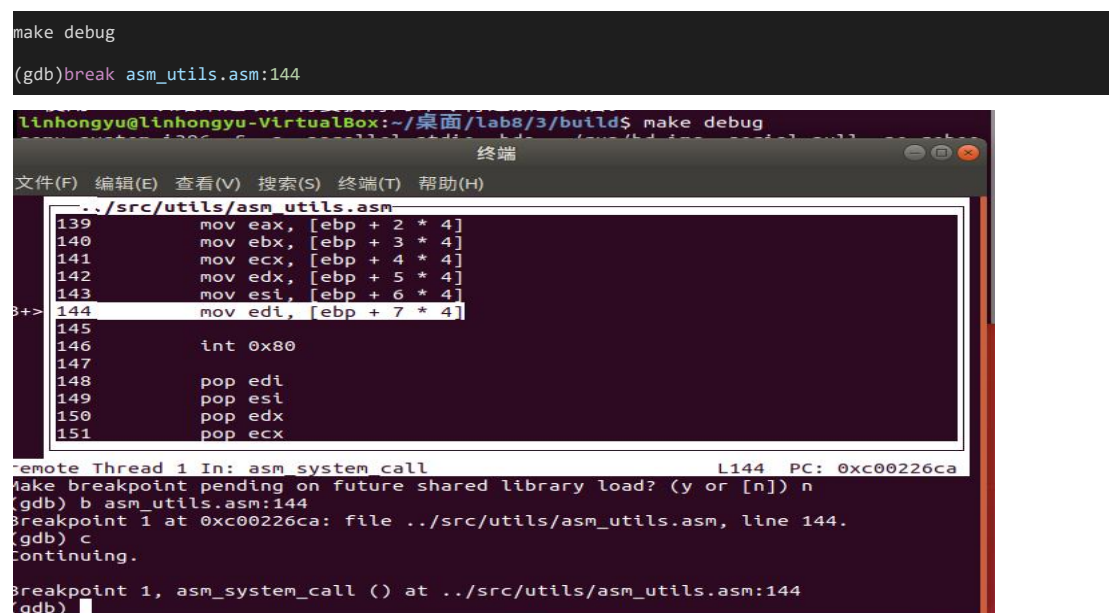
系统调用函数执行完毕后，返回值放入 eax 中。接着内核恢复原先保存的寄存器现场，并使用 `iret` 指令将控制权交还给用户态。通过这种机制，参数得以从用户栈传入寄存器，再由内核栈传给系统调用函数，实现从用户态到内核态的完整调用链。

2. 请使用 gdb 来分析在我们调用了 `int 0x80` 后，系统的栈发生了怎样的变化？esp 的值和在 `setup.cpp` 中定义的变量 `tss` 有什么关系？此外还有哪些段寄存器发生了变化？变化后的内容是什么？

2.1 调用了 `int 0x80` 后，系统的栈变化

2.1.1 启动调试

```
make debug
(gdb)break asm_utils.asm:144
```



```
linhongyu@linhongyu-VirtualBox:~/桌面/lab8/3/build$ make debug
make: *** No rule to make target 'asm_utils.asm', needed by 'asm_utils.o'.  Stop.
```

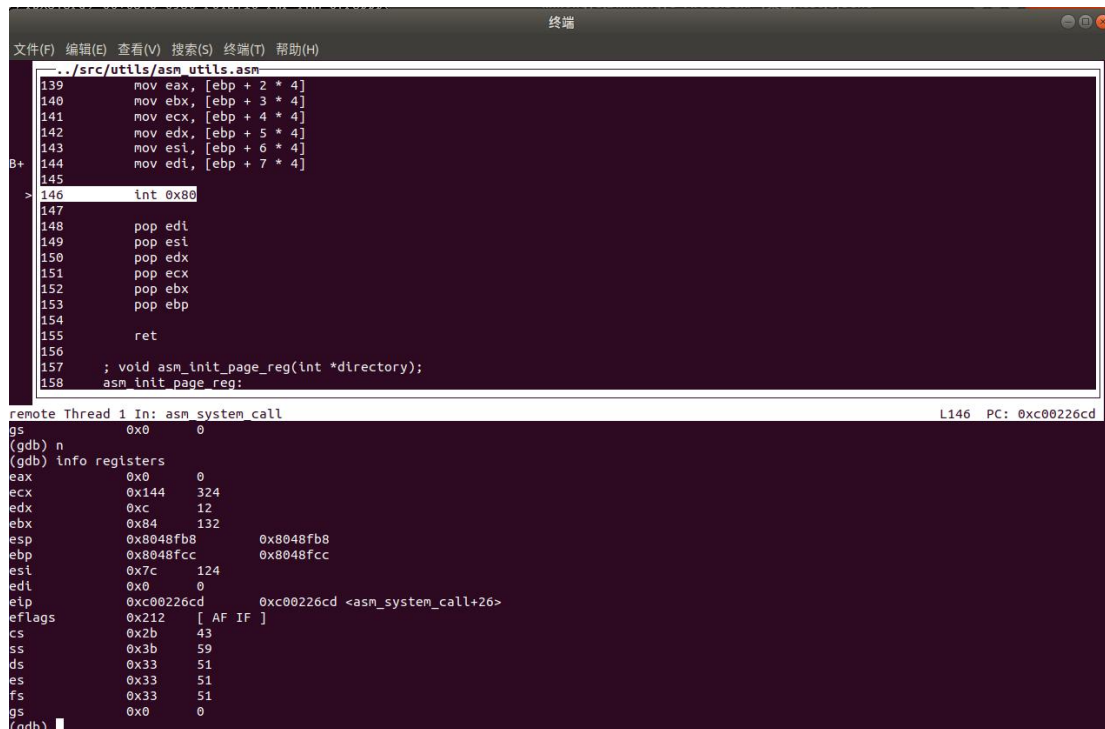
```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
~/src/utils/asm_utils.asm
139      mov eax, [ebp + 2 * 4]
140      mov ebx, [ebp + 3 * 4]
141      mov ecx, [ebp + 4 * 4]
142      mov edx, [ebp + 5 * 4]
143      mov esi, [ebp + 6 * 4]
3+> 144      mov edi, [ebp + 7 * 4]
145
146      int 0x80
147
148      pop edi
149      pop esi
150      pop edx
151      pop ecx

remote Thread 1 In: asm_system_call L144 PC: 0xc00226ca
make breakpoint pending on future shared library load? (y or [n]) n
(gdb) b asm_utils.asm:144
Breakpoint 1 at 0xc00226ca: file ../src/utils/asm_utils.asm, line 144.
(gdb) c
Continuing.

Breakpoint 1, asm_system_call () at ../src/utils/asm_utils.asm:144
(gdb)
```

2.1.2 观察栈寄存器变化

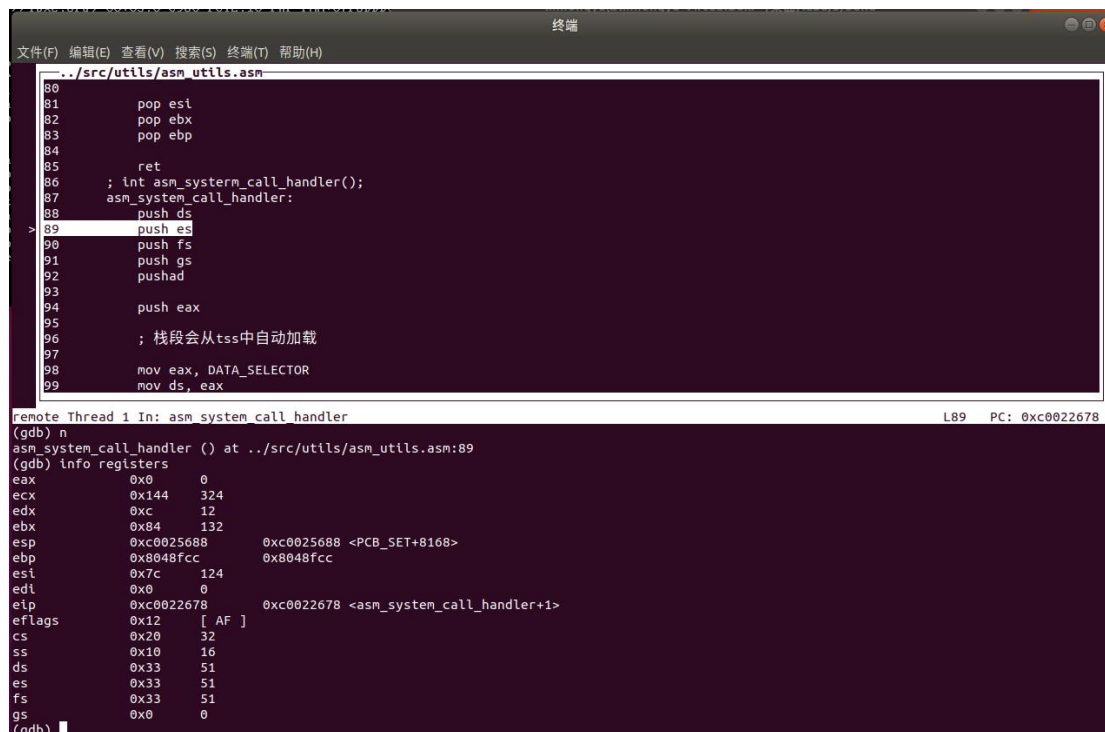
执行前栈寄存器内容如下：



The screenshot shows a debugger window with the title "终端". The assembly code in the main window is from `../src/utlis/asm_utils.asm` and includes instructions for pushing registers onto the stack and then popping them back. The instruction list shows lines 139 to 158. The instruction at line 146, `int 0x80`, is highlighted. Below the assembly code, the "remote Thread 1 In: asm_system_call" window shows the current instruction pointer (PC) at `L146` with the value `0xc00226cd`. The "(gdb) info registers" window displays the current state of the registers:

Register	Value	Comment
eax	0x0	0
ecx	0x144	324
edx	0xc	12
ebx	0x84	132
esp	0x8048fb8	0x8048fb8
ebp	0x8048fcc	0x8048fcc
esi	0x7c	124
edi	0x0	0
eip	0xc00226cd	0xc00226cd <asm_system_call+26>
eflags	0x212	[AF IF]
cs	0x2b	43
ss	0x3b	59
ds	0x33	51
es	0x33	51
fs	0x33	51
gs	0x0	0

执行后栈寄存器内容如下：

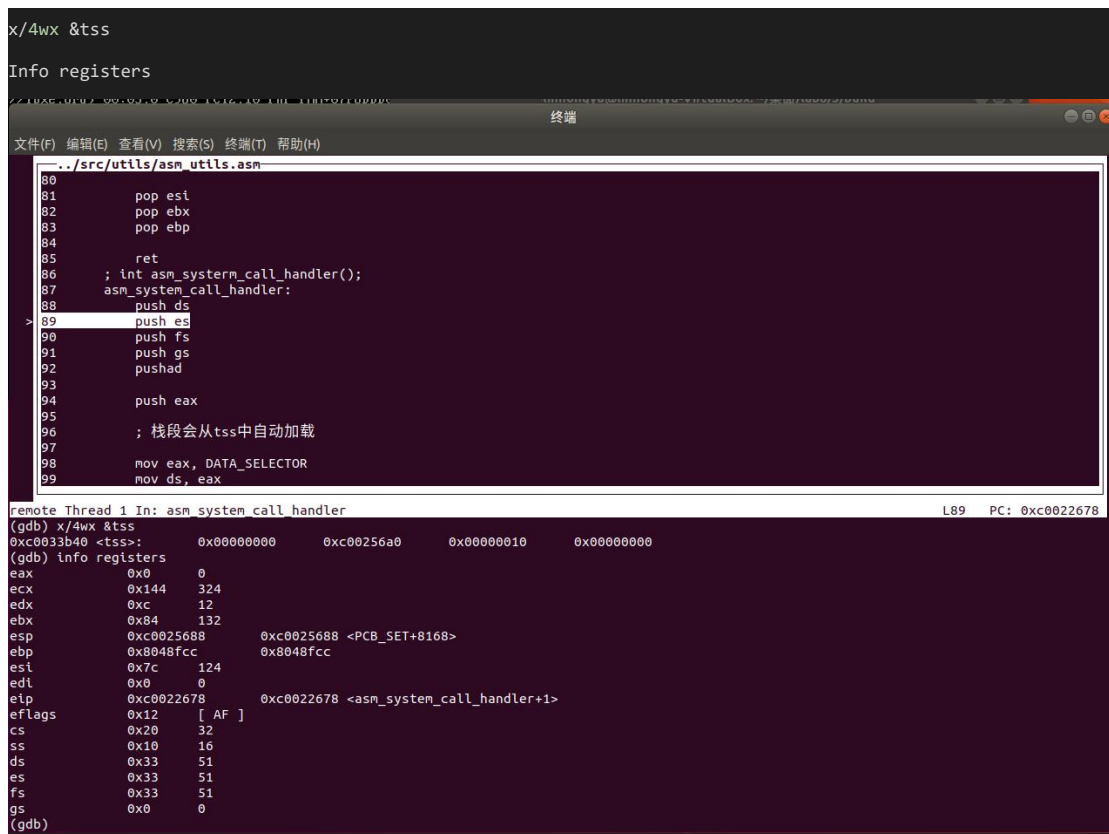


The screenshot shows the same debugger window after execution has proceeded. The assembly code in the main window now shows lines 80 to 99. The instruction at line 89, `push es`, is highlighted. The "remote Thread 1 In: asm_system_call_handler" window shows the current instruction pointer (PC) at `L89` with the value `0xc0022678`. The "(gdb) info registers" window displays the updated state of the registers:

Register	Value	Comment
eax	0x0	0
ecx	0x144	324
edx	0xc	12
ebx	0x84	132
esp	0xc0025688	0xc0025688 <PCB_SET+8168>
ebp	0x8048fcc	0x8048fcc
esi	0x7c	124
edi	0x0	0
eip	0xc0022678	0xc0022678 <asm_system_call_handler+1>
eflags	0x12	[AF]
cs	0x20	32
ss	0x10	16
ds	0x33	51
es	0x33	51
fs	0x33	51
gs	0x0	0

寄存器 `esp` 发生了变化，从用户态切换到了内核态

2.2 esp 的值和在 setup.cpp 中定义的变量 tss 关系



The screenshot shows a debugger window with two main panes. The top pane displays assembly code from a file named `../src/utils/asm_utils.asm`. The code includes instructions like `pop esi`, `pop ebx`, `pop ebp`, `ret`, `push ds`, `push es`, `push fs`, `push gs`, `pushad`, `push eax`, and `mov ds, eax`. A comment indicates that the stack segment is automatically loaded from the TSS. The bottom pane shows the state of the processor registers. The `esp` register is at `0xc0025688`, which is `<PCB_SET+8168>`. The `esp0` register is at `0xc00256a0`. The `ss` register is at `0x10`, indicating the kernel stack segment selector. The `ds`, `es`, and `fs` registers are at `0x33`, and the `gs` register is at `0x0`.

```
x/4wx &tss
Info registers
remote Thread 1 In: asm system call handler
(gdb) x/4wx &tss
0xc0033b40 <tss>: 0x00000000 0xc00256a0 0x00000010 0x00000000
(gdb) info registers
eax      0x0
ecx      0x144
edx      0xc
ebx      0x84
esp      0xc0025688 0xc0025688 <PCB_SET+8168>
ebp      0x8048fcc 0x8048fcc
esi      0x7c
edi      0x0
eip      0xc0022678 0xc0022678 <asm_system_call_handler+1>
eflags   0x12 [ AF ]
cs       0x20
ss       0x10
ds       0x33
es       0x33
fs       0x33
gs       0x0
(gdb)
```

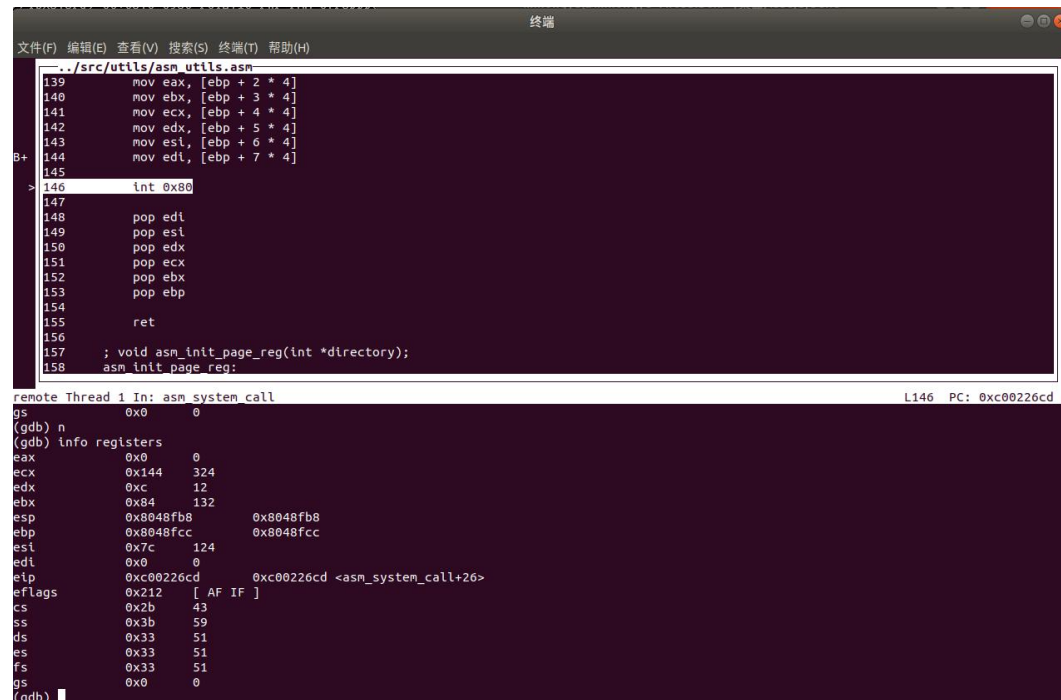
x/4wx \$tss 输出的第二个地址：0xc00256a0（esp0）是内核栈指针，tss.esp0 是内核栈的起始栈顶地址，int 0x80 执行后 CPU 自动将栈指针切换为 tss.esp0 指定的内核栈。

实际 esp 会比 esp0 小一些，因为 CPU 自动将当前寄存器现场和中断现场压入内核栈，导致 esp 下降。从 gdb 可以看到 esp 和 tss.esp0 非常接近，说明内核态栈正确切换成功。

tss 中的 ss0（0x10）表示内核栈的段选择子，内核栈段寄存器 ss 切换为 0x10

2.3 段寄存器发生了变化及其变化后的内容

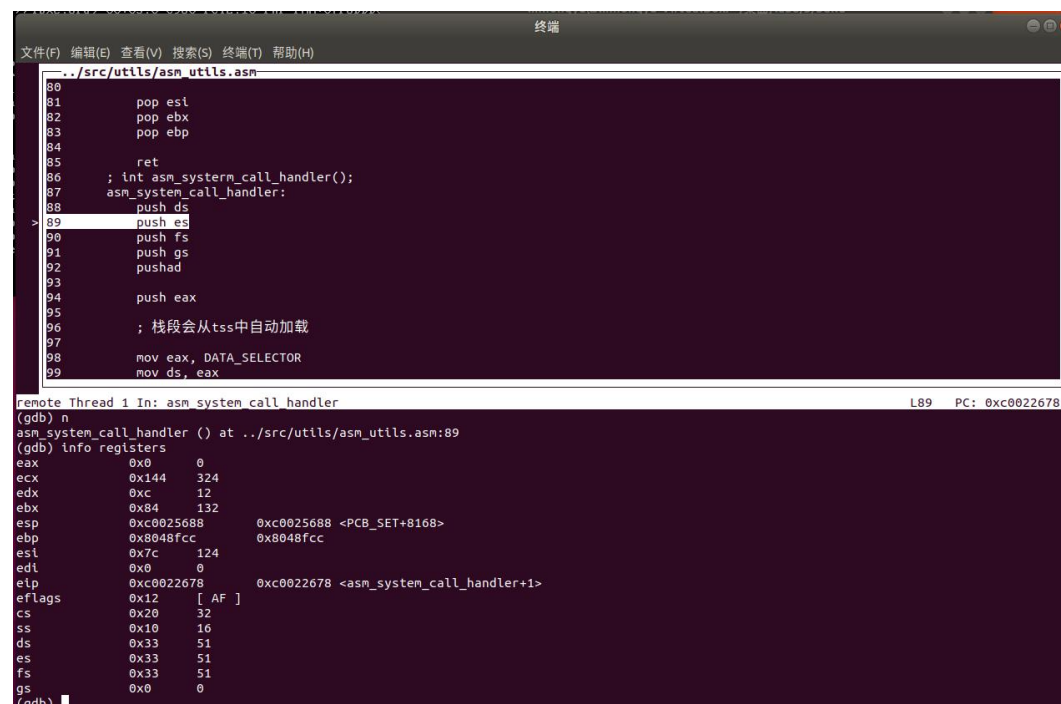
执行前栈寄存器内容如下：



```
..src/utlis/asm_utils.asm
139     mov eax, [ebp + 2 * 4]
140     mov ebx, [ebp + 3 * 4]
141     mov ecx, [ebp + 4 * 4]
142     mov edx, [ebp + 5 * 4]
143     mov esi, [ebp + 6 * 4]
144     mov edi, [ebp + 7 * 4]
145
> 146     int 0x80
147
148     pop edi
149     pop esi
150     pop edx
151     pop ecx
152     pop ebx
153     pop ebp
154
155     ret
156
157     ; void asm_init_page_reg(int *directory);
158     asm_init_page_reg:

remote Thread 1 In: asm_system_call
gs 0x0 0
(gdb) n
(gdb) info registers
eax 0x0 0
ecx 0x144 324
edx 0xc 12
ebx 0x84 132
esp 0x8048fb8 0x8048fb8
ebp 0x8048fcc 0x8048fcc
esi 0x7c 124
edi 0x0 0
eip 0xc00226cd 0xc00226cd <asm_system_call+26>
eflags 0x212 [ AF IF ]
cs 0x2b 43
ss 0x3b 59
ds 0x33 51
es 0x33 51
fs 0x33 51
gs 0x0 0
(gdb)
```

执行后栈寄存器内容如下：



```
..src/utlis/asm_utils.asm
80
81     pop esi
82     pop ebx
83     pop ebp
84
85     ret
86
87     ; int asm_system_call_handler();
88     asm_system_call_handler:
89     push ds
90     push esi
91     push fs
92     pushad
93
94     push eax
95
96     ; 栈段会从tss中自动加载
97
98     mov eax, DATA_SELECTOR
99     mov ds, eax

remote Thread 1 In: asm_system_call_handler
L89 PC: 0xc0022678
(gdb) n
asm_system_call_handler () at ../src/utlis/asm_utils.asm:89
(gdb) info registers
eax 0x0 0
ecx 0x144 324
edx 0xc 12
ebx 0x84 132
esp 0xc0025688 0xc0025688 <PCB_SET+8168>
ebp 0x8048fcc 0x8048fcc
esi 0x7c 124
edi 0x0 0
eip 0xc0022678 0xc0022678 <asm_system_call_handler+1>
eflags 0x12 [ AF ]
cs 0x20 32
ss 0x10 16
ds 0x33 51
es 0x33 51
fs 0x33 51
gs 0x0 0
(gdb)
```

段寄存器变化

cs: 0x2b 43——>0x20 32

ss: 0x3b 59——>0x10 16

3. 请使用 gdb 来分析在进入 `asm_system_call_handler` 的那一刻，栈顶的地址是什么？栈中存放的内容是什么？为什么存放的是这些内容？

3.1 栈顶的地址和栈中的内容

```
b asm_system_call_handler
c
x/16wx $esp

Info registers
(gdb) info registers
eax            0x0          0
ecx            0x144       324
edx            0xc         12
ebx            0x84        132
esp            0xc002568c   0xc002568c <PCB_SET+8172>
ebp            0xb048fcc    0xb048fcc
esi            0x7c        124
edi            0x0          0
eip            0xc0022677   0xc0022677 <asm_system_call_handler>
eflags         0x12        [ AF ]
cs             0x20        32
ss             0x10        16
ds             0x33        51
es             0x33        51
fs             0x33        51
gs             0x0          0
(gdb)
```

栈顶的地址(esp 指针): `esp=0xc002568c`

3.2 栈中内容

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
../src/utls/asm_utls.asm
82     pop ebx
83     pop ebp
84
85     ret
86     ; int asm_system_call_handler();
87     asm_system_call_handler:
88     push ds
89     push es
90     push fs
91     push gs
92     pushad
93
94     push eax
95
96     ; 栈段会从tss中自动加载
97
98     mov eax, DATA_SELECTOR
99     mov ds, eax
100    mov es, eax
101
remote Thread 1 In: asm_system_call_handler L88 PC: 0xc0022677
Undefined command: "z". Try "help".
(gdb) x/16wx $esp
0xc002568c <PCB_SET+8172>: 0xc00226cf  0x0000002b  0x00000212  0x00004fb8
0xc002569c <PCB_SET+8188>: 0x0000003b  0xc0026640  0x83e58955  0xec8308ec
0xc00256ac <PCB_SET+8204>: 0x00000a08  0x00000000  0x00000000  0x00000002
0xc00256bc <PCB_SET+8220>: 0x00000001  0x00000002  0x0000000a  0x00000000
```

3.3 为什么栈中存放这些内容

这些内容是由处理器在执行 `int 0x80` 指令时自动压入内核栈的。因为这是从用户态切换到内核态的过程，属于中断机制的一部分，CPU 会自动将当前用户态的执行现场保存下来，以便在中断处理完毕后能够通过 `iret` 指令正确地返回用户态继续执行。

具体来说，当 `int 0x80` 执行后，处理器首先将用户态的 `eip`（也就是中断发生时下一条要执行的指令地址）压入栈中，然后依次压入用户态的代码段选择子 `cs`、中断发生时的 `eflags` 值，以及用户态的栈顶地址 `esp` 和对应的栈段选择子 `ss`。

这些值构成了从用户态切换到内核态所需保存的最基本执行环境信息。

这是因为 `iret` 返回用户态时需要恢复这五个信息（`eip`、`cs`、`eflags`、`esp` 和 `ss`），才能让程序回到中断发生前的用户态位置继续执行。因此这些内容必须由 CPU 自动压栈，并且始终以固定顺序存在于栈中。我们在分析中看到的这些值，正是这段机制作用的结果。

4. 请结合代码分析 `asm_system_call_handler` 是如何找到中断向量号 `index` 对应的函数的

4.1 代码

```
asm_system_call_handler:
    push ds
    push es
    push fs
    push gs
    pushad

    push eax
    ; 栈段会从 tss 中自动加载
    mov eax, DATA_SELECTOR
    mov ds, eax
    mov es, eax
    mov eax, VIDEO_SELECTOR
    mov gs, eax
    pop eax
    ; 参数压栈
    push edi
    push esi
    push edx
    push ecx
    push ebx
    sti
    call dword[system_call_table + eax * 4]
    cli
    add esp, 5 * 4

    mov [ASM_TEMP], eax
    popad
    pop gs
    pop fs
    pop es
    pop ds
```

```
mov eax, [ASM_TEMP]
```

```
iret
```

4.2 分析

4.2.1 系统调用号存储在 `eax` 中

当用户程序执行 `int 0x80` 进入系统调用时，它会事先将调用号（比如调用 `syscall_0` 就是编号 0）保存在 `eax` 寄存器中。这个调用号表示要调用哪个系统调用函数。

4.2.2 构造函数表索引

系统调用处理函数保存在一个函数指针数组中，叫做 `system_call_table`，每个元素占 4 个字节。系统调用号作为索引，通过：

```
call dword [system_call_table + eax * 4]
```

4.2.3 直接调用系统调用函数

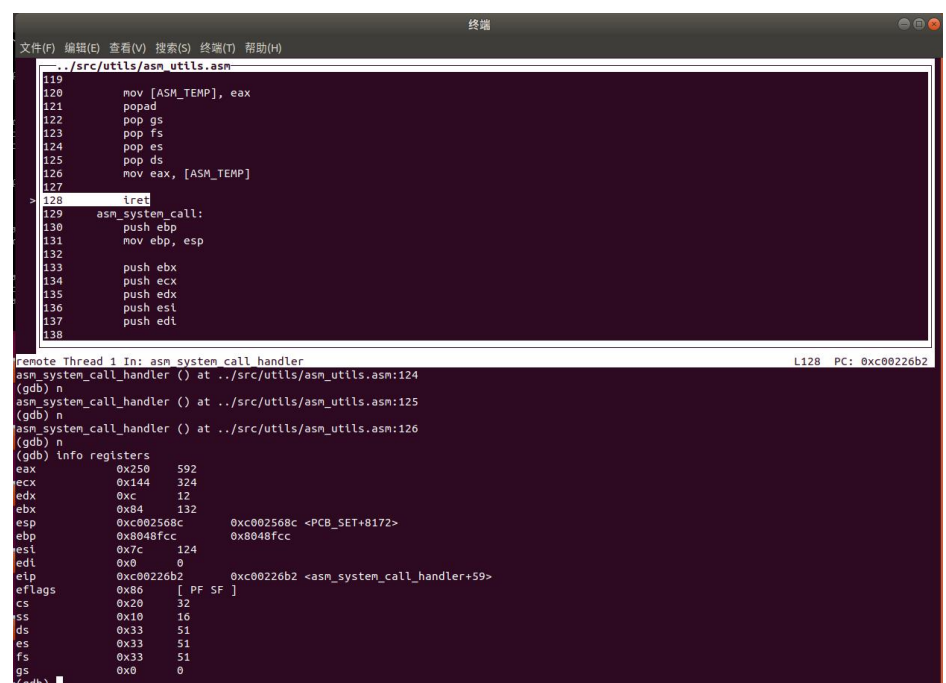
`call dword [...]` 是间接调用指令，会跳转执行系统调用函数本身。例如，如果 `eax == 0`，就会执行 `call dword [system_call_table + 0]`，也就是调用 `syscall_0`。

4.2.4 函数执行与返回值处理

被调用的系统调用函数在 C 层实现，并将其返回值放入 `eax` 中。由于之后还会有 `popad` 恢复寄存器，为防止 `eax` 被覆盖，先将其保存到 `ASM_TEMP` 中，等现场恢复后再将其重新装入 `eax` 返回给用户。

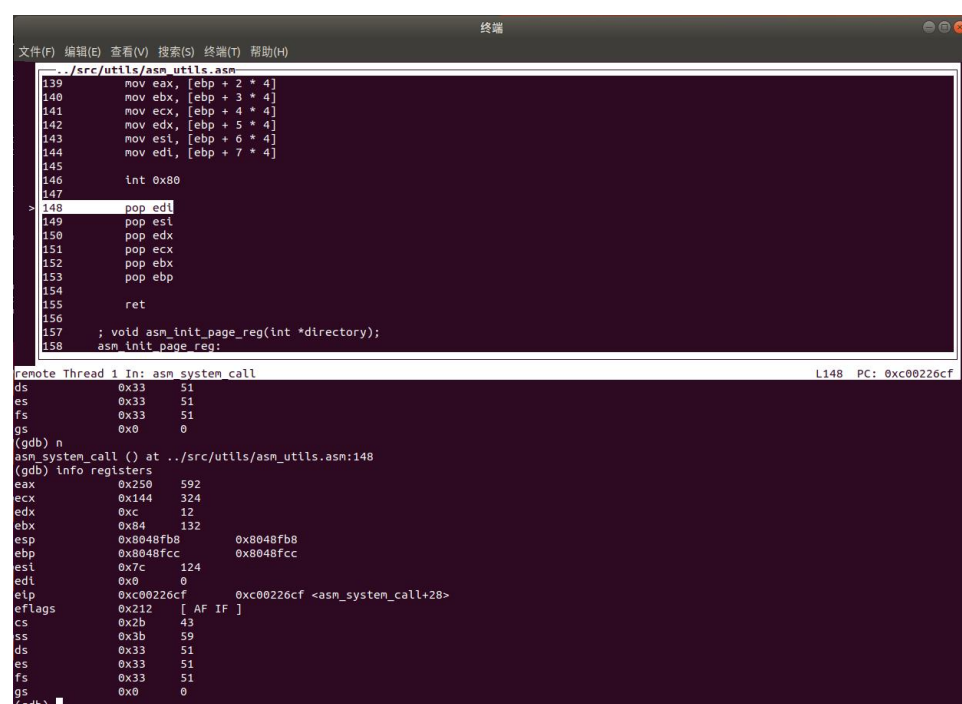
5. 请使用 gdb 来分析在 `asm_system_call` handler 中执行 `iret` 后，哪些段寄存器发生了变化？变化后的内容是什么？这些内容来自于什么地方？

5.1 段寄存器内容变化以及变化内容



```
.../src/utlis/asm_utils.asm
119      mov [ASM_TEMP], eax
120      popad
121      pop gs
122      pop fs
123      pop es
124      pop ds
125      pop ds
126      mov eax, [ASM_TEMP]
127
> 128      irt
129      asm_system_call:
130      push ebp
131      mov ebp, esp
132
133      push ebx
134      push ecx
135      push edx
136      push esi
137      push edi
138

remote Thread 1 In: asm system call handler
asm_system_call_handler () at ../src/utlis/asm_utils.asm:124
(gdb) n
asm_system_call_handler () at ../src/utlis/asm_utils.asm:125
(gdb) n
asm_system_call_handler () at ../src/utlis/asm_utils.asm:126
(gdb) n
(gdb) info registers
eax             0x250      592
ecx             0x144      324
edx             0xc        12
ebx             0x84       132
esp             0xc002568c  0xc002568c <PCB_SET+8172>
ebp             0x8048fcc   0x8048fcc
esi             0x7c       124
edi             0x0        0
eip             0xc00226b2  0xc00226b2 <asm_system_call_handler+59>
eflags         0x86       [ PF SF ]
cs              0x20       32
ss              0x10       16
ds              0x33       51
es              0x33       51
fs              0x33       51
gs              0x0        0
(gdb)
```



```
.../src/utlis/asm_utils.asm
139      mov eax, [ebp + 2 * 4]
140      mov ebx, [ebp + 3 * 4]
141      mov ecx, [ebp + 4 * 4]
142      mov edx, [ebp + 5 * 4]
143      mov esi, [ebp + 6 * 4]
144      mov edi, [ebp + 7 * 4]
145
146      int 0x80
147
> 148      pop edi
149      pop esi
150      pop edx
151      pop ecx
152      pop ebx
153      pop ebp
154      ret
155
156
157      ; void asm_init_page_reg(int *directory);
158      asm_init_page_reg:

remote Thread 1 In: asm system call
ds              0x33       51
es              0x33       51
fs              0x33       51
gs              0x0        0
(gdb) n
asm_system_call () at ../src/utlis/asm_utils.asm:148
(gdb) info registers
eax             0x250      592
ecx             0x144      324
edx             0xc        12
ebx             0x84       132
esp             0x8048fb8   0x8048fb8
ebp             0x8048fcc   0x8048fcc
esi             0x7c       124
edi             0x0        0
eip             0xc00226cf  0xc00226cf <asm_system_call+28>
eflags         0x212      [ AF IF ]
cs              0x2b       43
ss              0x3b       59
ds              0x33       51
es              0x33       51
fs              0x33       51
gs              0x0        0
(gdb)
```

变化的段寄存器：

cs：从 0x20（内核态）——> 0x2b（用户态代码段选择子）

ss：从 0x10（内核态）——> 0x3b（用户态栈段选择子）

5.2 内容来自什么地方

当用户程序通过 `int 0x80` 进入内核时，CPU 会自动将以下内容压入内核栈（由 TSS 中的 `esp0` 和 `ss0` 指定）：`ss` (用户态)、`esp` (用户态)、`eflags`、`cs` (用户态)、`eip` (用户态)，`iret` 会依次弹出 `eip`, `cs`, `eflags`, `esp`, `ss`，用于恢复到中断前用户态的执行现场。

执行 `iret` 后，`cs` 和 `ss` 发生变化，内容来自 `int 0x80` 时由 CPU 自动压入内核栈的用户态返回现场（`eip`、`cs`、`eflags`、`esp`、`ss`）。这正是特权级从 0 切换回 3（内核 → 用户）的关键操作，确保中断返回后恢复用户程序的正常执行

- 实验结果展示：在过程中已经呈现，不再赘述

----- 实验任务 2 -----

- 任务要求：进程的创建和调度。

复现“进程的实现”，“进程的调度”，“第一个进程”三节，并回答以下问题：

1. 请结合代码分析我们是如何在线程的基础上创建进程的 PCB 的(即分析进程创建的三个步骤)
2. 在进程的 PCB 第一次被调度执行时，进程实际上并不是跳转到进程的第一条指令处，而是跳转到 `load process`。请结合代码逻辑和 `gdb` 来分析为什么 `asm_switch_thread` 在执行 `iret` 后会跳转到 `load process`。
3. 在跳转到 `load process` 后，我们巧妙地设置了 `ProcessStartStack` 的内容，然后在 `asm_start_process` 中跳转到进程第一条指令处执行。请结合代码逻辑和 `gdb` 来分析我们是如何设置 `ProcessStartStack` 的内容，从而使得我们能够在 `asm_start_process` 中实现内核态到用户态的转移，即从特权级 0 转移到特权级 3 下，并使用 `iret` 指令成功启动进程的。
4. 结合代码，分析在创建进程后，我们对 `ProgramManager::schedule` 作了哪些修改？这样做的目的是什么？
5. 在进程的创建过程中，我们存在如下语句：

```
int ProgramManager::executeProcess(const char *filename, int priority)
{
    ...
    //找到刚刚创建的 PCB
    PCB *process = ListItem2PCB(allPrograms.back(), tagInAllList);
    ...
}
```

```
}
```

正如教程中所提到，” ……但是，这样做是存在风险的，我们应该通过 pid 来找到刚刚创建的 PCB。……”。现在，同学们需要编写一个 ProgramManager 的成员函数 findProgramByPid：

```
PCB *findProgramByPid(int pid);
```

并用上面这个函数替换指导书中提到的"存在风险的语句"，替换结果如下：

```
int ProgramManager::executeProcess(const char *filename, int priority)
{
    ...
    //找到刚刚创建的 PCB
    PCB *process =findProgramByPid(pid);
    ...
}
```

自行测试通过后，说一说你的实现思路，并保存结果截图。

● 实验步骤：

1. 请结合代码分析我们是如何在线程的基础上创建进程的 PCB 的(即分析进程创建的三个步骤)

1.1 创建进程的 PCB

```
int pid = executeThread((ThreadFunction)load_process,
                        (void *)filename, filename, priority);
```

使用已有的 executeThread 机制创建 PCB，底层仍沿用线程框架。唯一的区别是传入的函数不是线程主函数，而是 load_process，它将进程加载为用户态运行。创建完成后，PCB 被添加到 allPrograms 队列中，可以通过 allPrograms.back() 获取到。

1.2 初始化进程的页目录表

```
process->pageDirectoryAddress = createProcessPageDirectory();
```

调用 ProgramManager::createProcessPageDirectory() 在 内核空间 分配一页内存作为该进程的 页目录表。拷贝内核空间中页目录的第 768~1022 项（映射 3GB - 4GB）到新目录表对应项，以便进程也能访问内核空间。第 1023 项指向自己（用于高地址虚拟映射页目录本身，便于修改页表）。

1.3 初始化进程的虚拟地址池

```
createUserVirtualPool(process);
```

构造从 USER_VADDR_START 到 0xc0000000 之间的用户虚拟空间映射管理。通过位图方式记录每页是否被占用。位图本身放在内核空间（受保护）

2. 在进程的 PCB 第一次被调度执行时，进程实际上并不是跳转到进程的第一条指令处，而是跳转到 `load process`。请结合代码逻辑和 `gdb` 来分析为什么 `asm_switch_thread` 在执行 `ret` 后会跳转到 `load process`。

2.1 题意理解

题目说的是“进程”的 PCB 第一次被调用，而不是线程。你分析的应该是操作系统初始化时临时创建的第一个线程被调度时的情况。题目问的是通过 `first_thread` 创建三个进程的 `pcb` 后，后续开始调用进程时的情况

2.2 Gdb 分析命令

```
c asm_switch_thread
c
b asm_switch_thread
c
info registers
x/20x $esp
```

在第二次进入 `asm_switch_thread` 才是线程被调度的过程。

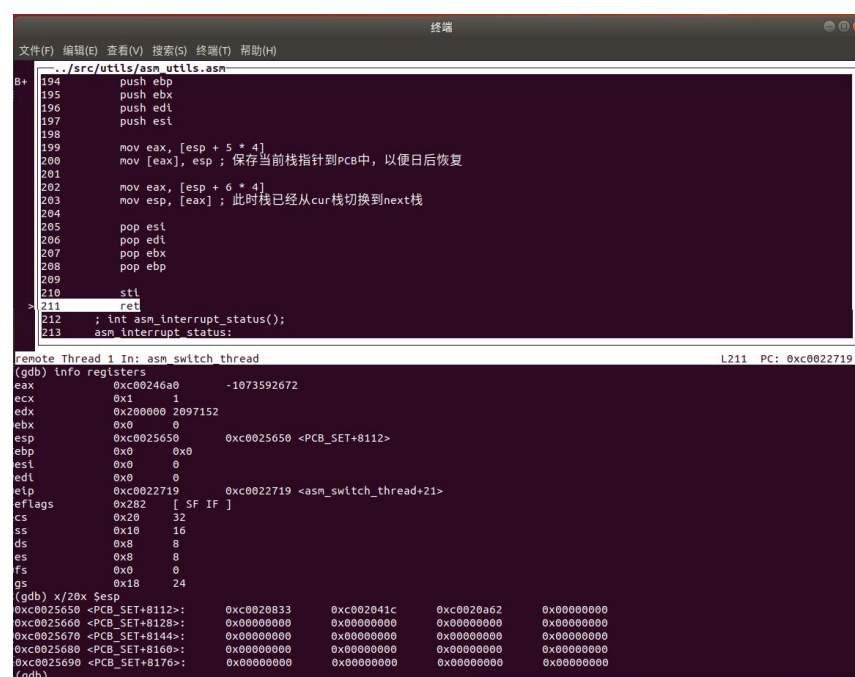
查看寄存器内容和 `esp` 指向的内容

2.3 调试结果

通过查看寄存器内容可以得知

栈顶 `esp` 指向的值会被 `ret` 弹出为返回地址

`ret` 前：

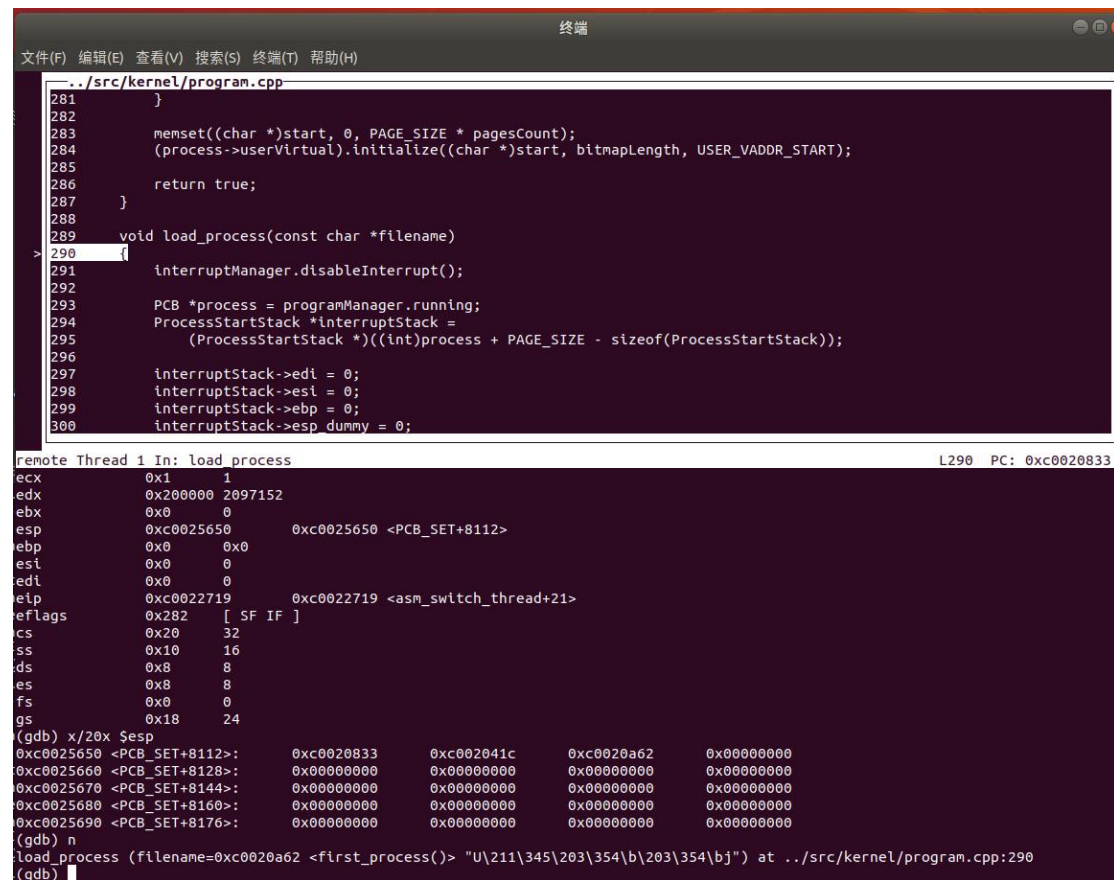


The screenshot shows a GDB terminal window with the following content:

```
remote Thread 1 In: asm_switch_thread
(gdb) info registers
eax      0xc00246a0    -1073592672
ecx      0x1          1
edx      0x200000     2097152
ebx      0x0          0
esp      0xc0025650    0xc0025650 <PCB_SET+8112>
ebp      0x0          0
esi      0x0          0
edi      0x0          0
eip      0xc0022719    0xc0022719 <asm_switch_thread+21>
eflags   0x282        [ SF IF ]
cs       0x20         32
ss       0x10         16
ds       0x8          8
es       0x8          8
fs       0x0          0
gs       0x18         24
(gdb) x/20x $esp
0xc0025650 <PCB_SET+8112>:  0xc0020833  0xc002041c  0xc0020a62  0x00000000
0xc0025660 <PCB_SET+8120>:  0x00000000  0x00000000  0x00000000  0x00000000
0xc0025670 <PCB_SET+8140>:  0x00000000  0x00000000  0x00000000  0x00000000
0xc0025680 <PCB_SET+8160>:  0x00000000  0x00000000  0x00000000  0x00000000
0xc0025690 <PCB_SET+8176>:  0x00000000  0x00000000  0x00000000  0x00000000
(gdb)
```

ret 后:

触发跳转到 PCB 中保存的初始函数 load_process。



```
../src/kernel/program.cpp
281     }
282
283     memset((char *)start, 0, PAGE_SIZE * pageCount);
284     (process->userVirtual).initialize((char *)start, bitmapLength, USER_VADDR_START);
285
286     return true;
287 }
288
289 void load_process(const char *filename)
290 {
291     interruptManager.disableInterrupt();
292
293     PCB *process = programManager.running;
294     ProcessStartStack *interruptStack =
295         (ProcessStartStack *)((int)process + PAGE_SIZE - sizeof(ProcessStartStack));
296
297     interruptStack->edi = 0;
298     interruptStack->esi = 0;
299     interruptStack->ebp = 0;
300     interruptStack->esp dummy = 0;
}

remote Thread 1 In: load_process L290 PC: 0xc0020833
ecx      0x1      1
edx      0x200000 2097152
ebx      0x0      0
esp      0xc0025650 0xc0025650 <PCB_SET+8112>
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
ebp      0xc0022719 0xc0022719 <asm_switch_thread+21>
eflags   0x282    [ SF IF ]
cs       0x20     32
ss       0x10     16
ds       0x8      8
es       0x8      8
fs       0x0      0
gs       0x18     24
(gdb) x/20x $esp
0xc0025650 <PCB_SET+8112>: 0xc0020833 0xc002041c 0xc0020a62 0x00000000
0xc0025660 <PCB_SET+8128>: 0x00000000 0x00000000 0x00000000 0x00000000
0xc0025670 <PCB_SET+8144>: 0x00000000 0x00000000 0x00000000 0x00000000
0xc0025680 <PCB_SET+8160>: 0x00000000 0x00000000 0x00000000 0x00000000
0xc0025690 <PCB_SET+8176>: 0x00000000 0x00000000 0x00000000 0x00000000
(gdb) n
load_process (filename=0xc0020a62 <first_process()> "U\211\345\203\354\b\203\354\bj") at ../src/kernel/program.cpp:290
(gdb)
```

2.4 代码理解

```
// 设置新线程的初始内核栈
thread->stack = (int *)((int)thread + PCB_SIZE - sizeof(ProcessStartStack));
thread->stack -= 7;
thread->stack[0] = 0;
thread->stack[1] = 0;
thread->stack[2] = 0;
thread->stack[3] = 0;
thread->stack[4] = (int)function; // <== function = load_process
thread->stack[5] = (int)program_exit;
thread->stack[6] = (int)parameter;
```

这段代码为新线程准备了一个 伪栈帧（fake stack frame），这个栈帧是供 ret 使用的，模拟了一个“调用 function（即 load_process）”返回地址在栈顶的场景。所以当 asm_switch_thread() 做完 mov esp, ... 后执行 ret，就会跳转到：

```
thread->stack[4] = (int)function == (int)load_process
```

并且它会使用 thread->stack[6] 作为参数传给 load_process，也就是文件名

3. 在跳转到 `load process` 后，我们巧妙地设置了 `ProcessStartStack` 的内容，然后在 `asm_start_process` 中跳转到进程第一条指令处执行。请结合代码逻辑和 `gdb` 来分析我们是如何设置 `ProcessStartStack` 的内容，从而使得我们能够在 `asm_start_process` 中实现内核态到用户态的转移，即从特权级 0 转移到特权级 3 下，并使用 `iret` 指令成功启动进程的。

3.1 ProcessStartStack

`load_process` 代码如下：

```
void load_process(const char *filename)
{
    interruptManager.disableInterrupt();

    PCB *process = programManager.running;
    ProcessStartStack *interruptStack =
        (ProcessStartStack *)((int)process + PAGE_SIZE - sizeof(ProcessStartStack));
    interruptStack->edi = 0;
    interruptStack->esi = 0;
    interruptStack->ebp = 0;
    interruptStack->esp_dummy = 0;
    interruptStack->ebx = 0;
    interruptStack->edx = 0;
    interruptStack->ecx = 0;
    interruptStack->eax = 0;
    interruptStack->gs = 0;
    interruptStack->fs = programManager.USER_DATA_SELECTOR;
    interruptStack->es = programManager.USER_DATA_SELECTOR;
    interruptStack->ds = programManager.USER_DATA_SELECTOR;
    interruptStack->eip = (int)filename;
    interruptStack->cs = programManager.USER_CODE_SELECTOR; // 用户模式平坦模式
    interruptStack->eflags = (0 << 12) | (1 << 9) | (1 << 1); // IOPL, IF = 1 开中断, MBS = 1 默认
    interruptStack->esp = memoryManager.allocatePages(AddressPoolType::USER, 1);
    if (interruptStack->esp == 0)
    {
        printf("can not build process!\n");
        process->status = ProgramStatus::DEAD;
        asm_halt();
    }
    interruptStack->esp += PAGE_SIZE;
    interruptStack->ss = programManager.USER_STACK_SELECTOR;
    asm_start_process((int)interruptStack);
}
```


ProcessStartStack 本质上是进程的内核态栈的起始内容（其实是进程切换时内核栈中的内容）。在内核向用户进程切换时，CPU 通过 **iret** 指令恢复上下文。**iret** 会依次从当前栈中弹出 **EIP**、**CS**、**EFLAGS**、**ESP**、**SS** 等寄存器值，实现特权级和代码段的切换。

因此，我们要提前构造这个栈帧，使得 **iret** 弹出后，CPU 能“自动”切换到用户态代码入口，并且切换特权级到 3。

3.2 设置 ProcessStartStack

```
ProcessStartStack *interruptStack =  
    (ProcessStartStack *)((int)process + PAGE_SIZE - sizeof(ProcessStartStack));
```

给当前进程 **process** 的内核栈空间（假设大小为 **PAGE_SIZE**，即 4KB）。**process** 指针是 PCB 基址，往上偏移 **PAGE_SIZE** 到栈顶，再减去 **ProcessStartStack** 结构体大小，放置伪造上下文栈帧。把 **ProcessStartStack** 放在该进程内核栈空间的栈顶。

3.3 使用 ProcessStartStack

在内核栈顶放置 **ProcessStartStack** 结构，伪造了完整的 CPU 上下文。其中包含了恢复现场必需的 **EIP**、**CS**、**EFLAGS**、**ESP**、**SS** 以及数据段寄存器和通用寄存器。这样 **iret** 执行后，CPU 自动切换到用户态，开始执行 **filename** 指向的程序入口，使用分配的用户栈。

3.4 gdb 调试

刚进入 **asm_start_process** 时：

进程还处于内核态（**CS=0x20**，**SS=0x10** 是内核段选择子）

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
../src/utlis/asm_utils.asm
36      mov cr3, eax
37      pop eax
38      ret
39      asm_start_process:
40          jmp $
B+> 41      mov eax, dword[esp+4]
42      mov esp, eax
43      popad
44      pop gs;
45      pop fs;
46      pop es;
47      pop ds;
48
49      iret
50
51      ; void asm_ltr(int tr)
52      asm_ltr:
53          ltr word[esp + 1 * 4]
54          ret
55
56      ; int asm_add_global_descriptor(int low, int high);
57      asm_add_global_descriptor:

remote Thread 1 In: asm start process L41 PC: 0xc0022620
(gdb) c
Continuing.

Breakpoint 1, asm_start_process () at ../src/utlis/asm_utils.asm:41
(gdb) info registers
eax            0xc002565c      -1073508644
ecx            0xc0010000      -1073676288
edx            0x3b          59
ebx            0x0            0
esp            0xc0025624      0xc0025624 <PCB_SET+8068>
ebp            0xc0025650      0xc0025650 <PCB_SET+8112>
esi            0x0            0
edi            0x0            0
eip            0xc0022620      0xc0022620 <asm_start_process>
eflags         0x92          [ AF SF ]
cs             0x20          32
ss             0x10          16
ds             0x8           8
es             0x8           8
fs             0x0           0
gs             0x18          24
(gdb)
```

执行 iret 前：

DS/ES/FS 切换到了用户数据段选择子 0x33，这是用户段选择子，说明即将从内核态切换到用户态。

ESP 指向伪造的 ProcessStartStack 栈顶（栈帧）。

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
../src/utlis/asm_utils.asm
36      mov cr3, eax
37      pop eax
38      ret
39      asm_start_process:
40          jmp $
B+> 41      mov eax, dword[esp+4]
42      mov esp, eax
43      popad
44      pop gs;
45      pop fs;
46      pop es;
47      pop ds;
48
49      iret
50
51      ; void asm_ltr(int tr)
52      asm_ltr:
53          ltr word[esp + 1 * 4]
54          ret
55
56      ; int asm_add_global_descriptor(int low, int high);
57      asm_add_global_descriptor:

remote Thread 1 In: asm start process L49 PC: 0xc002262d
(gdb) n
(gdb) n
(gdb) n
(gdb) n
(gdb) info registers
eax            0x0            0
ecx            0x0            0
edx            0x0            0
ebx            0x0            0
esp            0xc002568c      0xc002568c <PCB_SET+8172>
ebp            0x0            0x0
esi            0x0            0
edi            0x0            0
eip            0xc002262d      0xc002262d <asm_start_process+13>
eflags         0x92          [ AF SF ]
cs             0x20          32
ss             0x10          16
ds             0x33          51
es             0x33          51
fs             0x33          51
gs             0x0            0
(gdb)
```

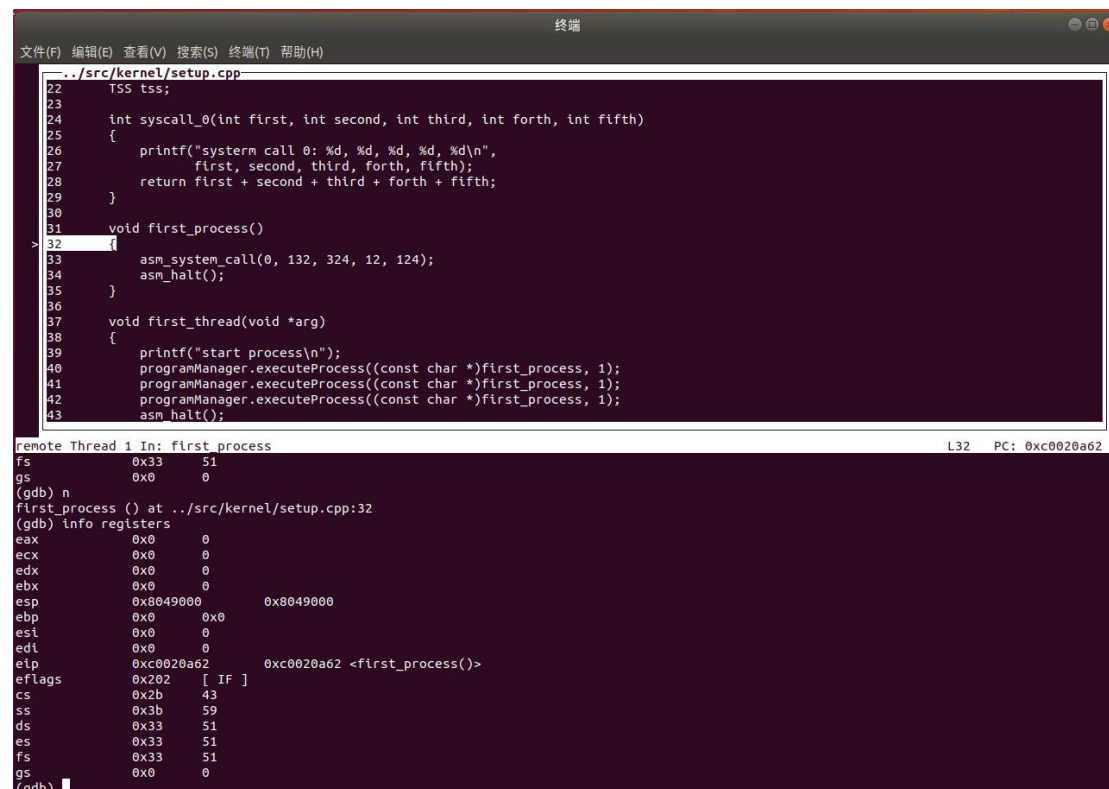
执行 `iret` 后：

CS 变成 0x2b, SS 变成 0x3b, 都是用户态的代码段和栈段选择子。

EIP 跳转到 `first_process()` 函数地址, 说明 `iret` 根据你伪造的 `ProcessStartStack` 中的 `eip` 恢复了用户代码入口地址。

ESP 变成了 0x8049000, 是你分配给用户栈的地址 (之前在 `load_process` 中分配的用户栈)。

EFLAGS 的值 0x202 表示中断标志开启。



```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/kernel/setup.cpp
22     TSS tss;
23
24     int syscall_0(int first, int second, int third, int forth, int fifth)
25     {
26         printf("system call 0: %d, %d, %d, %d, %d\n",
27               first, second, third, forth, fifth);
28         return first + second + third + forth + fifth;
29     }
30
31     void first_process()
32     {
33         asm_system_call(0, 132, 324, 12, 124);
34         asm_halt();
35     }
36
37     void first_thread(void *arg)
38     {
39         printf("start process\n");
40         programManager.executeProcess((const char *)first_process, 1);
41         programManager.executeProcess((const char *)first_process, 1);
42         programManager.executeProcess((const char *)first_process, 1);
43         asm_halt();
44     }

remote Thread 1 In: first process
fs 0x33 51
gs 0x0 0
(gdb) n
first_process () at ../src/kernel/setup.cpp:32
(gdb) info registers
eax 0x0 0
ecx 0x0 0
edx 0x0 0
ebx 0x0 0
esp 0x8049000 0x8049000
ebp 0x0 0x0
esi 0x0 0
edi 0x0 0
eip 0xc0020a62 0xc0020a62 <first_process()>
eflags 0x202 [ IF ]
cs 0x2b 43
ss 0x3b 59
ds 0x33 51
es 0x33 51
fs 0x33 51
gs 0x0 0
(gdb)
```

4. 结合代码, 分析在创建进程后, 我们对 `ProgramManager::schedule` 作了哪些修改? 这样做的目的是什么?

4.1 代码

```
void ProgramManager::activateProgramPage(PCB *program)
{
    int paddr = PAGE_DIRECTORY;

    if (program->pageDirectoryAddress)
    {
        tss.esp0 = (int)program + PAGE_SIZE;
        paddr = memoryManager.vaddr2paddr(program->pageDirectoryAddress);
    }
}
```

```
asm_update_cr3(paddr);  
}
```

4.2 修改

4.2.1 更新 tss.esp0

每个进程都有一片独立的内核栈（位于 PCB 顶部，大小=PAGE_SIZE）。在从用户态（CPL=3）到内核态（CPL=0）的中断或系统调用发生时，CPU 会自动读取 TSS 中的 esp0/ss0，把它当作内核态栈。所以在切到 next 进程时，必须把 tss.esp0 指向它的内核栈顶。

4.2.2 切换 CR3（页目录寄存器）

只有切换了 CR3，CPU 才会使用 next->pageDirectoryAddress 对应的页表去做地址变换，从而让用户态代码访问到自己独立的虚拟空间。否则，所有进程都会共用内核的页目录，彼此间无法隔离。

4.3 目的

4.3.1 进程间内存隔离

每个进程都有独立的页目录表和页表，切换 CR3 保证它们访问到自己的虚拟空间，互不干扰。

4.3.2 支持用户态和内核态之间切换

当用户进程执行到系统调用或被时钟中断打断时，CPU 会自动根据 TSS 中的 esp0/ss0 切换到内核栈。正确更新 tss.esp0，才能让内核在服务下一个用户进程的时候，用它自己的内核栈，不会压到别的进程的栈上。

4.3.3 维护调度一致性

将虚拟地址空间和内核栈切换集成到同一个调度函数里，使得不管是线程（共享内核页目录）还是进程（独立页目录），都能通过统一的 schedule() 完成上下文切换。

5. 同学们需要编写一个 ProgramManager 的成员函数 findProgramByPid

5.1 为什么原语句存在风险

5.2 修改后的代码

5.2.1 成员函数

```
PCB *ProgramManager::findProgramByPid(int pid)  
{  
    ListItem *item = allPrograms.front();
```

```

while (item)
{
    PCB *program = ListItem2PCB(item, tagInAllList);

    if (program->pid == pid)
    {
        return program;
    }

    item = item->next;
}

return nullptr; // 如果没找到
}

```

5.2.2 替换原语句

原来的语句：

```
PCB *process = ListItem2PCB(allPrograms.back(), tagInAllList);
```

替换后的语句：

```
PCB *process = findProgramByPid(pid);
```

5.2.3 记得在.h 文件说明



```

// 执行线程调度
void schedule();

// 阻塞唤醒
void MESA_WakeUp(PCB *program);

// 初始化TSS
void initializeTSS();

// 创建线程
int executeProcess(const char *filename, int priority);

// 创建用户项目录表
int createProcessPageDirectory();

// 创建用户地址池
bool createUserVirtualPool(PCB *process);

// 切换项目录表，实现虚拟地址空间的切换
void activateProgramPage(PCB *program);

PCB *findProgramByPid(int pid);

};

void program_exit();
void load_process(const char *filename);

#endif

```

5.3 运行结果

```

make
make run

```

```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab8/findProgramByPid/build
QEMU

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
system call 0: 132, 324, 12, 124, 0
system call 0: 132, 324, 12, 124, 0
system call 0: 132, 324, 12, 124, 0
```

- 实验结果展示：结果已经在过程中逐步展示了。

----- 实验任务 3 -----

- 任务要求：fork 的实现

复现“fork”一小节的内容，并回答以下问题：

1. 请根据代码逻辑概括 fork 的实现的思路，并简要分析我们是如何解决四个关键问题“的。
2. 请根据 gdb 来分析子进程第一次被调度执行时，即在 asm_switch_thread 切换到子进程的栈中时，esp 的地址是什么？栈中保存的内容是什么？
3. 从子进程第一次被调度执行时开始，逐步跟踪子进程的执行流程一直到子进程从 fork 返回，根据 gdb 来分析子进程的跳转地址、数据寄存器和段寄存器的变化。同时，比较上述过程和父进程执行完 ProgramManager::fork 后的返回过程的异同。
4. 请根据代码逻辑和 gdb 来解释子进程的 fork 返回值为什么是 0，而父进程的 fork 返回值是子进程的 pid。
5. 请解释在 Programanager::schedule 中，我们是如何从一个进程的虚拟地址空间切换到另外一个进程的虚拟地址空间的。

- 实验步骤：

1. 请根据代码逻辑概括 `fork` 的实现的基本思路，并简要分析我们是如何解决"四个关键问题"的。

`fork` 的实现四个关键问题

1.1 如何实现父子进程的代码段共享？

所有进程使用相同的内核代码段，内核映射在每个进程的 3GB-4GB 虚拟空间顶部，实现了天然共享。

1.2 如何使得父子进程从相同的返回点开始执行？

`fork` 时复制了父进程的 0 特权级栈（`ProgramStartStack`），其中保存了中断发生时的寄存器现场和返回地址。子进程通过 `asm_start_process` 恢复执行上下文，从而从 `fork()` 返回点继续执行。

1.3 除代码段外，进程包含的资源有哪些？

包括 PCB、0 级栈、虚拟地址位图、用户空间页目录、页表、用户内存页等。

1.4 如何实现进程的资源在进程之间的复制？

使用临时的中转页：先从父进程复制到内核中转页，再切换到子进程页表，从中转页复制到子进程虚拟空间，实现资源复制而不发生虚拟地址冲突。

总结 `fork` 实现思路

父进程发起 `fork` 系统调用，内核通过 `ProgramManager::fork` 实现新进程的创建。创建一个新的 PCB，并复制父进程的状态与资源。

使用 `copyProcess` 完成父进程资源的复制，包括 0 特权级栈、用户虚拟地址池、页目录、页表等。

设置子进程初始的内核栈结构，使其能从 `asm_start_process` 正确开始执行。

返回父进程子进程的 PID，子进程返回 0。

2. 请根据 gdb 来分析子进程第一次被调度执行时，即在 `asm_switch_thread` 切换到子进程的栈中时，`esp` 的地址是什么？栈中保存的内容是什么？

`esp` 表示当前线程内核栈的栈顶指针，寄存器内容如图可见：

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/utlis/asm_utils.asm
203    mov esp, [eax] ; 此时栈已经从cur栈切换到next栈
204
205    pop esi
206    pop edi
207    pop ebx
208    pop ebp
209
210    sti
211    ret
> 211    ret
212    ; int asm_interrupt_status();
213    asm_interrupt_status:
214    xor eax, eax
215    pushfd
216    pop eax
217    and eax, 0x200
218    ret
219
220    ; void asm_disable_interrupt();
221    asm_disable_interrupt:

remote Thread 1 In: asm_switch_thread L211 PC: 0xc0022d19
asm_switch_thread () at ../src/utlis/asm_utils.asm:211
(gdb) info registers
eax      0xc0023de0      -1073594912
ecx      0xc0023de0      -1073594912
edx      0x0             0
ebx      0x0             0
esp      0xc0024d90      0xc0024d90 <PCB_SET+4016>
ebp      0x0             0x0
esi      0x0             0
edi      0x0             0
eip      0xc0022d19      0xc0022d19 <asm_switch_thread+21>
eflags   0x212          [ AF IF ]
cs       0x20            32
ss       0x10            16
ds       0x8             8
es       0x8             8
fs       0x0             0
gs       0x18            24
(gdb)
```

3. 从子进程第一次被调度执行时开始，逐步跟踪子进程的执行流程一直到子进程从 fork 返回，根据 gdb 来分析子进程的跳转地址、数据寄存器和段寄存器的变化。同时，比较上述过程和父进程执行完 ProgramManager::fork 后的返回过程的异同。

跟踪子进程创建流程：

3.1 fork 调用函数

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/kernel/syscall.cpp
25    }
26
27    int write(const char *str) {
28        return asm_system_call(1, (int)str);
29    }
30
31    int syscall_write(const char *str) {
32        return stdio.print(str);
33    }
34
35    int fork() {
36        return asm_system_call(2);
37    }
38
39    int syscall fork() {
40        return programManager.fork();
41    }^?
42
43
44

remote Thread 1 In: syscall fork L40 PC: 0xc0020f79
0x0000ffff in ?? ()
(gdb) b syscall.cpp:40
Breakpoint 1 at 0xc0020f79: file ../src/kernel/syscall.cpp, line 40.
(gdb) c
Continuing.
Breakpoint 1, syscall_fork () at ../src/kernel/syscall.cpp:40
(gdb)
```

3.2 在函数体内

判断当前是进程还是线程

如果是进程的话就分配新的 PCB


```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
../src/kernel/setup.cpp
31 void first_process()
32 {
33     int pid = fork();
34
35     if (pid == -1)
36     {
37         printf("can not fork\n");
38     }
39     else
40     {
41         if (pid)
42         {
43             printf("I am father, fork reutr: %d\n", pid);
44         }
45         else
46         {
47             printf("I am child, fork return: %d, my pid: %d\n", pid, programManager.running->pid);
48         }
49     }
50 }
```

3.3 复制函数内容

操作中复制了 0 级栈，并将子进程寄存器集合中的 `eax` 设为 0；按照启动流程构造子进程的栈，以确保调度切换时能正常跳转至启动函数；复制 PCB 中的状态、优先级、进程名及父进程的 PID，为父进程原路返回提供依据；复制用户虚拟地址池及父进程的页表，相当于同时继承了其他特权级的栈结构。

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
../src/kernel/program.cpp
378
379 bool ProgramManager::copyProcess(PCB *parent, PCB *child)
380 {
381     // 复制进程0级栈
382     ProcessStartStack *childpss =
383         (ProcessStartStack *)((int)child + PAGE_SIZE - sizeof(ProcessStartStack));
384     ProcessStartStack *parentpss =
385         (ProcessStartStack *)((int)parent + PAGE_SIZE - sizeof(ProcessStartStack));
386     memcpy(parentpss, childpss, sizeof(ProcessStartStack));
387     // 设置子进程的返回值为0
388     childpss->eax = 0;
389
390     // 准备执行asm_switch_thread的栈的内容
391     child->stack = (int *)childpss - 7;
392     child->stack[0] = 0;
393     child->stack[1] = 0;
394     child->stack[2] = 0;
395     child->stack[3] = 0;
396     child->stack[4] = (int)asm_start_process;
}

remote Thread 1 In: ProgramManager::copyProcess L388 PC: 0xc0020af9
```

3.4 返回 pid，父进程结束

`iret` 指令不仅用于返回，还会完整恢复中断或异常前的执行上下文：包括弹出返回地址到 `CS:EIP`、恢复 `EFLAGS`（如中断标志 `IF`），必要时还会执行任务状态段（`TSS`）切换，从而实现权限级别的切换与控制。

这里结束一步步退出，就完成了一次子进程的创建。在父进程中，我们得到的子进程的 `pid`，然后一路一路传下来，最终就得到了父进程的 `pid`。

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/kernel/program.cpp
362     }
363
364     // 初始化子进程
365     PCB *child = ListItem2PCB(this->allPrograms.back(), tagInAllList);
366     bool flag = copyProcess(parent, child);
367
368     if (!flag)
369     {
370         child->status = ProgramStatus::DEAD;
371         interruptManager.setInterruptStatus(status);
372         return -1;
373     }
374
375     interruptManager.setInterruptStatus(status);
376     return pid;
377 }
378
379 bool ProgramManager::copyProcess(PCB *parent, PCB *child)
380 {
    remote Thread 1 In: ProgramManager::fork
    (gdb) n
    L375 PC: 0xc0020aaf
```

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/utils/asm_utils.asm
120     mov [ASM_TEMP], eax
121     popad
122     pop gs
123     pop fs
124     pop es
125     pop ds
126     mov eax, [ASM_TEMP]
127
128     iret
129     asm_system_call:
130     push ebp
131     mov ebp, esp
132
133     push ebx
134     push ecx
135     push edx
136     push esi
137     push edi
138
    remote Thread 1 In: asm_system_call_handler
    (gdb) n
    (gdb) n
    (gdb) n
    L128 PC: 0xc0020cb2
```

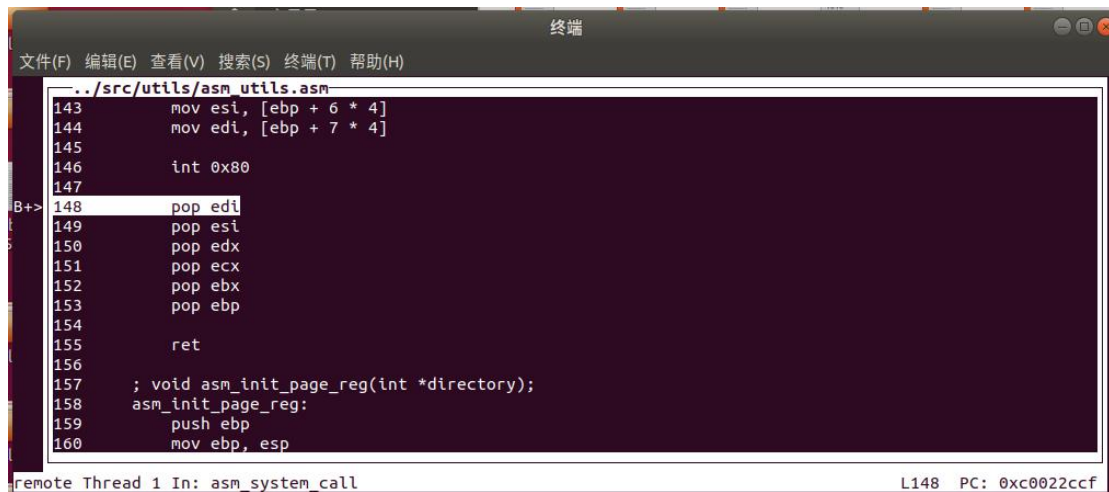
3.5 切换子进程（依靠中断）

```
终端
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

../src/kernel/program.cpp
120     PCB *next = ListItem2PCB(item, tagInGeneralList);
121     PCB *cur = running;
122     next->status = ProgramStatus::RUNNING;
123     running = next;
124     readyPrograms.pop_front();
125
126     //printf("schedule: %x %x\n", cur, next);
127
128     activateProgramPage(next);
129
130     asm_switch_thread(cur, next);
131
132     interruptManager.setInterruptStatus(status);
133 }
134
135 void program_exit()
136 {
137     PCB *thread = programManager.running;
138     thread->status = ProgramStatus::DEAD;
    remote Thread 1 In: ProgramManager::schedule
    L130 PC: 0xc00203f4
```

3.6 接着跟踪 `asm_switch_thread` 和 `asm_start_process`

3.7 然后回到 `int 0x80` 后



```
..../src/utlis/asm_utils.asm
143     mov esi, [ebp + 6 * 4]
144     mov edi, [ebp + 7 * 4]
145
146     int 0x80
147
B+> 148     pop edi
149     pop esi
150     pop edx
151     pop ecx
152     pop ebx
153     pop ebp
154
155     ret
156
157     ; void asm_init_page_reg(int *directory);
158     asm_init_page_reg:
159         push ebp
160         mov ebp, esp

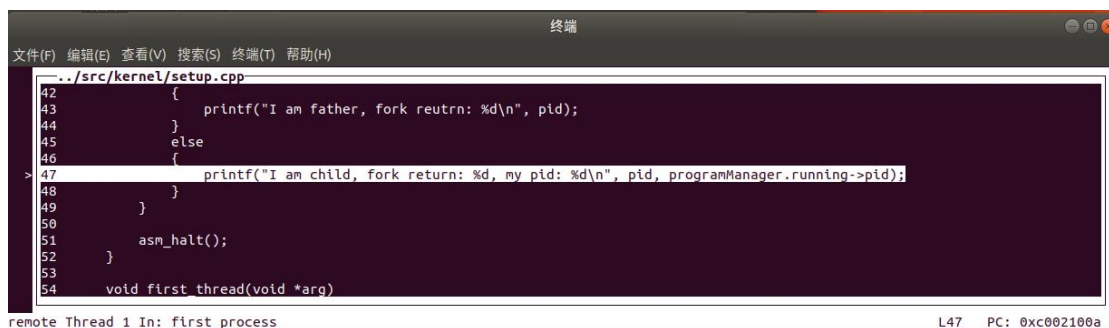
remote Thread 1 In: asm_system_call L148 PC: 0xc0022ccf
```

子进程继承了父进程在 fork 时的完整执行上下文，未清空代码和栈，且我们仅将返回值寄存器 `eax` 设为 0，其他寄存器保持不变。因此，子进程从与父进程相同的位置继续执行，fork 返回值为 0。验证时发现执行正好停在 `int 0x80` 后，说明除了 `eax`，其余寄存器和返回地址完全一致。

3.8 回到 setup 函数

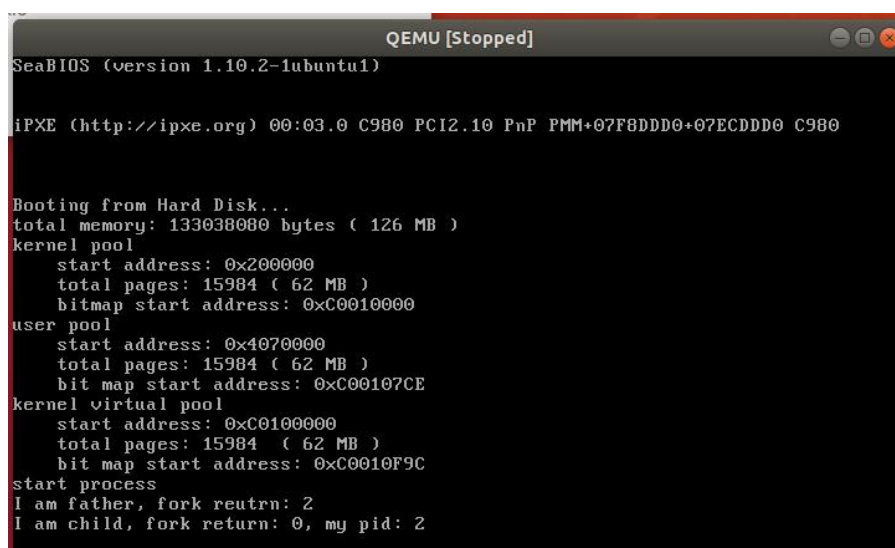
行完这里，函数一定会返回到 `setup` 中的判断 `pid` 是否为 0 的函数中。

判断 `pid` 输出如下结果：



```
..../src/kernel/setup.cpp
42     {
43         printf("I am father, fork reutrn: %d\n", pid);
44     }
45     else
46     {
47         printf("I am child, fork return: %d, my pid: %d\n", pid, programManager.running->pid);
48     }
49 }
50
51     asm_halt();
52 }
53
54 void first_thread(void *arg)

remote Thread 1 In: first_process L47 PC: 0xc002100a
```



```
QEMU [Stopped]
SeaBIOS (version 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
I am father, fork reutrn: 2
I am child, fork return: 0, my pid: 2
```

4. 请根据代码逻辑和 gdb 来解释子进程的 fork 返回值为什么是 0，而父进程的 fork 返回值是子进程的 pid。

4.1 copyProcess 中复制父进程内核栈时专门设置了：childpss->eax = 0；这个字段对应的是进程执行 iret 后 CPU 恢复的 EAX 值，因此子进程中 fork() 返回值为 0。

4.2 父进程调用 ProgramManager::fork，该函数的返回值为 pid（子进程 PID），所以 fork() 返回的是子进程 pid。

5. 请解释在 Programanager::schedule 中，我们是如何从一个进程的虚拟地址空间切换到另外一个进程的虚拟地址空间的。

5.1 调度器选中下一个进程 next

5.2 调用 activateProgramPage(next);

5.3 设置新进程的内核栈（0 特权级栈）

5.4 设置新的页目录表

5.5 CPU 立即使用新的页目录进行地址转换，实现虚拟地址空间切换

● 实验结果展示：结果已经在过程中展示，不再赘述。

----- 实验任务 4 -----

● 任务要求：wait & exit 的实现

参考指导书中“wait”和“exit”两节的内容，实现 wait 函数和 exit 函数，回答以下问题：

1. 请结合代码逻辑和具体的实例来分析 exit 的执行过程。
2. 请解释进程退出后能够隐式调用 exit 的原因。(tips:从栈的角度分析)
3. 请结合代码逻辑和具体的实例来分析 wait 的执行过程。
4. 如果一个父进程先于子进程退出，那么子进程在退出之前会被称为孤儿进程。子进程在退出后，从状态被标记为 DEAD 开始到被回收，子进程会被称为僵尸进程。请对代码做出修改，实现回收僵尸进程的有效方法。

● 实验步骤:

1. 请结合代码逻辑和具体的实例来分析 `exit` 的执行过程。

1.1 代码

调用逻辑

```
exit(ret) → asm_system_call(3, ret) → syscall_exit(ret) → ProgramManager::exit(ret)
```

1.1.1 标记状态 + 保存返回值

```
exit(ret) → asm_system_call(3, ret) → syscall_exit(ret) → ProgramManager::exit(ret)
PCB *program = this->running;
program->retValue = ret;
program->status = ProgramStatus::DEAD;
```

当前运行进程/线程的 PCB 状态设为 DEAD。

返回值保存在 `PCB::retValue` 中，用于后续进程/线程回收。

1.1.2 释放资源（仅对进程）

```
if (program->pageDirectoryAddress) { ... }
```

如果当前是“进程”：

遍历页目录和页表，释放用户空间所映射的所有物理页。

释放页目录本身的物理页。

释放虚拟地址空间的位图占用内存。

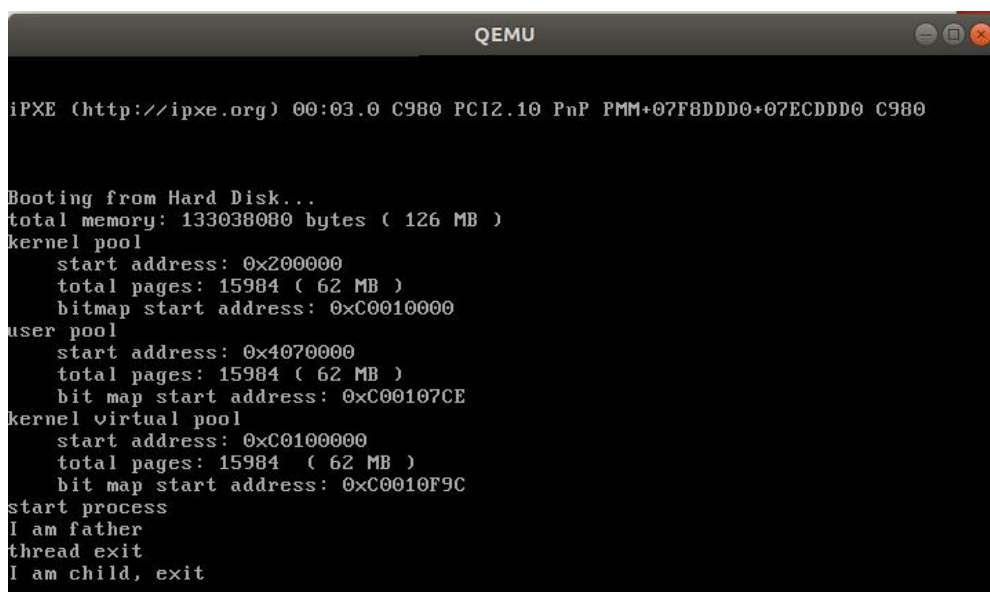
（线程不会有独立页目录，因此跳过此步。）

1.1.3 立即调度下一个进程/线程

```
schedule();
```

当前运行实体标记为 DEAD 后，调度器从就绪队列中选择下一个进程/线程运行。

1.2 示例分析



```
QEMU

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD0+07ECDDDD0 C980

Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x2000000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
I am father
thread exit
I am child, exit
```

```

if (pid) {
    printf("I am father\n");
    asm_halt();
} else {
    printf("I am child, exit\n");
}

```

在 child 进程中打印完后未显式调用 `exit(0)`，但仍然会自动退出，这就引出了第二问中说的“隐式调用 `exit`”

2. 请解释进程退出后能够隐式调用 `exit` 的原因。(tips:从栈的角度分析)

2.1 `load_process` 中构造了用户栈帧

```

int *userStack = (int *)interruptStack->esp;
userStack -= 3;
userStack[0] = (int)exit; // return address → exit
userStack[1] = 0;         // dummy return address of exit
userStack[2] = 0;         // exit 参数 ret
interruptStack->esp = (int)userStack;

```

2.2 原因

本质上，`ret` 指令通过弹栈决定跳转位置，因此只要在进程启动前按调用顺序将返回地址压入栈中，进程结束时就能自然跳转。虽然 `exit` 中未显式赋值返回值，但函数参数也是通过栈传递的。创建进程时，我们在栈中构造了 `exit` 的调用帧，并默认传入参数 0，因此 `ret` 返回值自然为 0。这种方式模拟了线程的返回机制，实现了隐式退出。

3. 请结合代码逻辑和具体的实例来分析 `wait` 的执行过程。

3.1 调用 `wait(int *retval)` 流程

会通过系统调用陷入内核，进入 `syscall_wait`，实际调用 `wait(retval)` 函数。

在内核中，父进程遍历系统中所有进程，寻找其子进程（`parentPid == running->pid`）。

如果找到状态为 `DEAD` 的子进程：读取其返回值（如果 `retval` 非空）、调用 `releasePCB(child)` 清理子进程 `PCB`、返回子进程的 `pid`。

如果找到了子进程但都没有处于 `DEAD` 状态，父进程被挂起调度（阻塞等待子进程结束）。

如果根本没有子进程，`wait` 返回 -1。

3.2 示例说明

```
void first_process() {
    int pid = fork();
    int retval;

    if (pid) {
        pid = fork();
        if (pid) {
            while ((pid = wait(&retval)) != -1) {
                printf("wait for a child process, pid: %d, return value: %d\n", pid, retval);
            }
            printf("all child process exit\n");
            asm_halt();
        }
        ...
    } else {
        ...
        exit(-123);
    }
}
```

first_process 创建两个子进程。

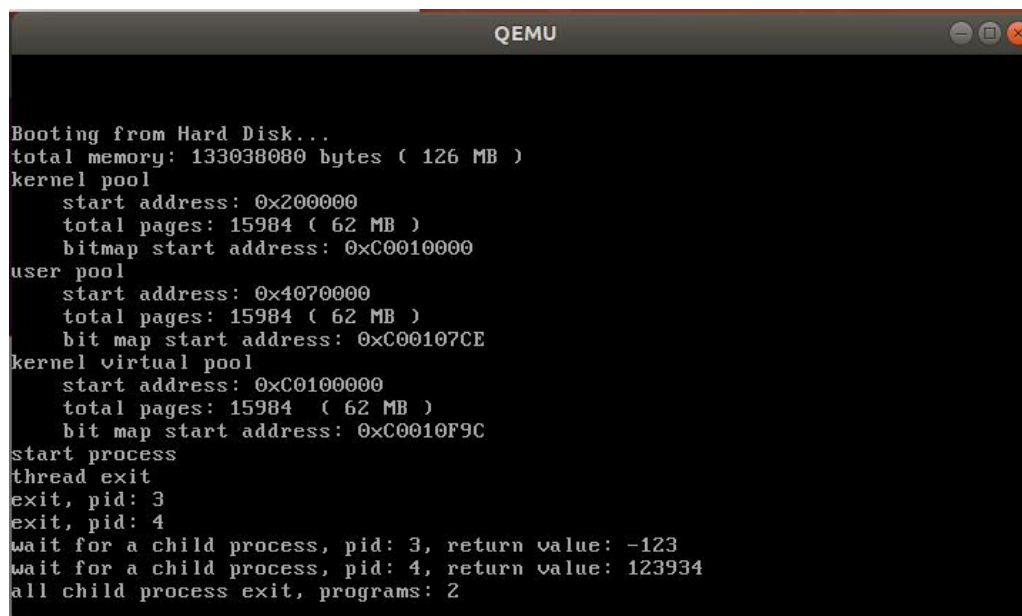
父进程调用 wait() 两次来等待两个子进程结束。

子进程退出时调用 exit(ret)，进入 DEAD 状态。

父进程从 wait() 中恢复执行，获取返回值并释放子进程 PCB。

如果所有子进程都已回收，则 wait() 返回 -1。

3.3 运行结果展示



The image shows a QEMU terminal window with the following output:

```
Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
thread exit
exit, pid: 3
exit, pid: 4
wait for a child process, pid: 3, return value: -123
wait for a child process, pid: 4, return value: 123934
all child process exit, programs: 2
```


4. 如果一个父进程先于子进程退出，那么子进程在退出之前会被称为孤儿进程。子进程在退出后，从状态被标记为 DEAD 开始到被回收，子进程会被称为僵尸进程。请对代码做出修改，实现回收僵尸进程的有效方法。

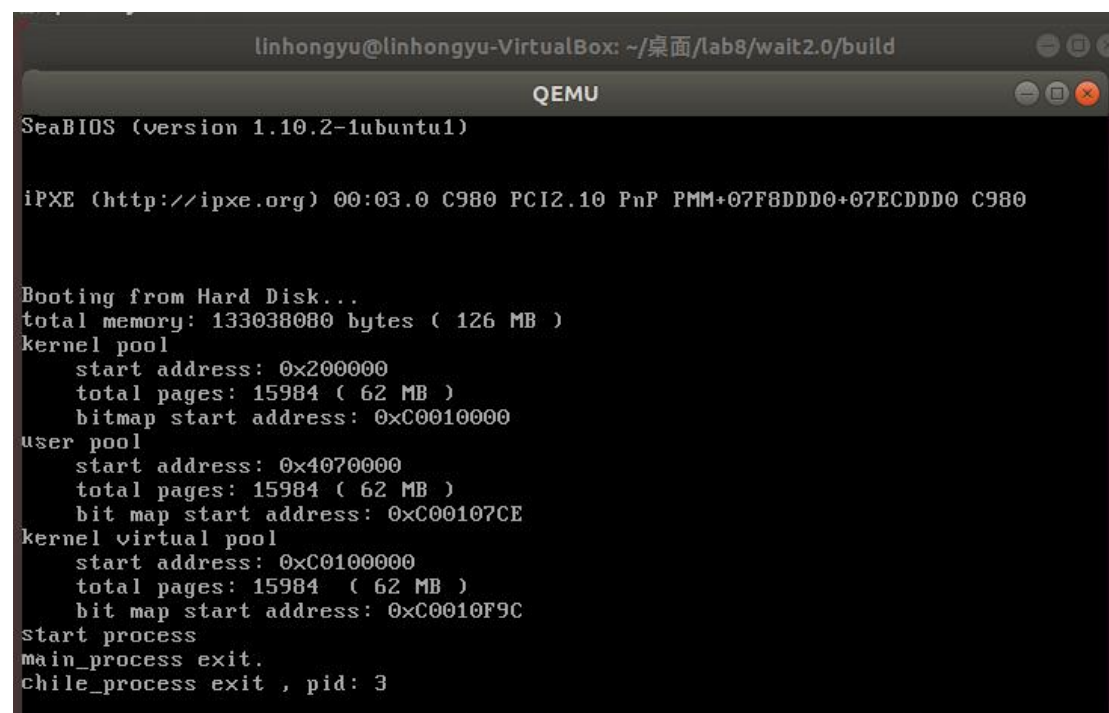
4.1 僵尸进程例子（修改 first_process）

```
void first_process()
{
    int pid = fork();
    int retval;

    if (pid)
    {
        printf("main_process exit.\n");
    }
    else
    {
        uint32 tmp = 0xffffffff;
        while (tmp)
            --tmp;

        printf("chile_process exit , pid: %d\n", programManager.running->pid);
        exit(-123);
    }
}
```

结果显示，主进程结束后，子进程才退出



```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab8/wait2.0/build
QEMU
SeaBIOS (version 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD0+07ECDDDD0 C980

Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0xC0010000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC00107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0xC0010F9C
start process
main_process exit.
chile_process exit , pid: 3
```


4.2 回收僵尸进程代码（修改 wait 函数）

```
int ProgramManager::wait(int *retval)
{
    PCB *child;
    ListItem *item;
    bool interrupt, flag;

    while (true)
    {
        interrupt = interruptManager.getInterruptStatus();
        interruptManager.disableInterrupt();
        item = this->allPrograms.head.next;
        flag = true;
        while (item)
        {
            {
                child = ListItem2PCB(item, tagInAlllist);
                if (child->parentPid)
                {
                    flag = false;
                    if (child->status == ProgramStatus::DEAD)
                    {
                        break;
                    }
                }
                item = item->next;
            }
        }
        if (item)
        {
            if (retval)
            {
                *retval = child->retValue;
            }
            int pid = child->pid;
            releasePCB(child);
            interruptManager.setInterruptStatus(interrupt);
            printf("child_process recycle , pid=%d",pid);
            return pid;
        }
        else
        {
            if (flag)
            {
                interruptManager.setInterruptStatus(interrupt);
                return -1;
            }
        }
    }
}
```

```

    }
    else
    {
        interruptManager.setInterruptStatus(interrupt);
        schedule();
    }
}
}
}
}
}
```

运行结果:

由结果可见，成功收回子进程。

```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab8/wait2.0/build
QEMU
SeaBIOS (version 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...
total memory: 133038080 bytes ( 126 MB )
kernel pool
    start address: 0x200000
    total pages: 15984 ( 62 MB )
    bitmap start address: 0xC0010000
user pool
    start address: 0x4070000
    total pages: 15984 ( 62 MB )
    bit map start address: 0xC00107CE
kernel virtual pool
    start address: 0xC0100000
    total pages: 15984 ( 62 MB )
    bit map start address: 0xC0010F9C
start process
main_process exit.
chile_process exit , pid: 3
child_process recycle , pid=3
```

Section 5 实验总结与心得体会

在本次实验课程中，我深入学习了操作系统的底层原理，掌握了汇编编程技巧，并积累了联合编译和大型项目开发的经验，这也是本学期最后一次实验，综合了这学期的所有知识点。

操作系统这门课为我打开了操作系统学习的大门。尽管操作系统本身庞大复杂，本课程仅覆盖了其中基础部分，但每一次实验都极具挑战性，需要我投入大量时间去理解每一行代码的意义。

操作系统实验的学习离不开指导书和老师、助教们的辛勤付出，常常在深夜 debug 时能够收到助教关于实验内容的答疑回复。

正是老师和助教们极度认真的态度，使得庞大的操作系统变得浅显易懂。在此向你们敬佩与感激。

以实验 8 的最后一个项目的例子做结

敲下“make run”会是什么呢？

...



linhongyu@linhongyu-VirtualBox: ~/桌面/lab8/Ending!!/build

文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

linhongyu@linhongyu-VirtualBox:~/桌面/lab8/Ending!!/build\$ make run

QEMU

SUMMER

Welcome to SUMMER Project!

<https://gitee.com/nelsoncheung/sysu-2021-spring-operating-system/>

SUMMER is an OS course project.

Proposed and led by
Prof. PengFei Chen.

Developed by
Prof. PengFei Chen,
HPC and AI students in grade 2019,
HongYang Chen,
WenXin Xie,
YuZe Fu,
Nelson Cheung.